Running surface-partition on Pelle

A Step-by-Step Guide for New Collaborators

May 2026

Abstract

This guide covers everything a new collaborator needs to go from zero to running **surface-partition** jobs on UPPMAX Pelle: obtaining cluster access, setting up the Python environment, cloning and configuring the repository, submitting Phase 1 relaxation jobs and Phase 2 refinement jobs, running parameter sweeps, and retrieving results for local analysis. No prior experience with Pelle or SLURM is assumed.

Contents

1 Prerequisites

Before starting, make sure you have the following:

- **UPPMAX account.** Request one via the SUPR portal (https://supr.naiss.se). Ask the project PI to add you to the allocation `uppmax2025-2-534` once your account is active.
- **SSH client.** On Linux and macOS the built-in `ssh` command is sufficient. On Windows, use Git Bash, WSL, or PuTTY.
- **Repository access.** You need read access to the GitHub repository. Ask the PI for the URL and confirm you can clone it before your first login to Pelle.
- **Basic command-line familiarity.** You should be comfortable navigating directories, editing text files, and running commands in a Linux shell. No prior HPC or SLURM experience is required.

2 Logging in to Pelle

Pelle is UPPMAX's newest cluster. Connect with SSH, replacing *<your_username>* with your UPPMAX username:

```
ssh <your_username>@pelle.uppmax.uu.se
```

The first connection will ask you to confirm the host fingerprint — type `yes` and press Enter.

Tip

SSH key authentication. Password prompts can get tedious. Set up key-based login once:

```
# On your local machine
ssh-keygen -t ed25519 -C "pelle"
ssh-copy-id <your_username>@pelle.uppmax.uu.se
```

After this, `ssh pelle.uppmax.uu.se` logs you in without a password.

3 Directory Layout on Pelle

UPPMAX uses a two-tier storage model. Understanding it now prevents confusion later about where files end up.

Location	Quota	Notes
`~/`	32 GB	Home directory. Backed up. Personal to each user.
`/proj/snic2020-15-36/private/`	Large	Project directory. **Shared** by all project members.

The conventions used by this project are:

- **Code, configuration, scripts** — live in your home directory under `~/projects/surface-partition/`. Each collaborator has their own copy.
- **Python virtual environment** — lives in your home directory under `~/venvs/surface-partition/`. Each collaborator has their own environment.
- **Computation results** (HDF5 solution files, refinement checkpoints, traces) — written to the shared project directory at `/proj/snic2020-15-36/private/LINKED_LST_MANIFOLD/results/`. All collaborators share this directory.

- **SLURM log files** — written inside the repository at `~/projects/surface-partition/slurm_logs/`. Personal to each user.

Important

The shared project directory `/proj/snic2020-15-36/private/` is **not backed up**. Do not use it as the only copy of important data. Transfer final results to your local machine or a backed-up location.

4 First-Time Setup

Perform these steps once on your first login. They take about ten minutes.

4.1 Create the directory structure

```
mkdir -p ~/projects
mkdir -p ~/venvs
```

4.2 Clone the repository

Clone the repository from GitHub:

```
cd ~/projects
git clone git@github.com:TebanVe/surface-partition.git surface-partition
```

Note

This uses SSH-based cloning, which requires your SSH public key to be registered on GitHub. If you have not done this yet, go to https://github.com/settings/keys and add the public key from Pelle (`~/.ssh/id_ed25519.pub` or similar). Alternatively, use the HTTPS URL `https://github.com/TebanVe/surface-partition.git` and authenticate with a personal access token.

This creates `~/projects/surface-partition/` with the full repository contents.

4.3 Load the Python module

Pelle uses the *module* system to manage software. Load Python with:

```
module load Python/3.11.5-GCCcore-13.3.0
```

Note

To see which Python versions are available: `module spider Python`. The project currently targets `Python/3.11.5-GCCcore-13.3.0`. If it is unavailable, contact the PI for an updated module name and update `cluster/pelle_config.sh` accordingly.

4.4 Create the Python virtual environment

```
python -m venv ~/venvs/surface-partition
source ~/venvs/surface-partition/bin/activate
pip install --upgrade pip
```

4.5 Install the project and its dependencies

```
cd ~/projects/surface-partition
pip install -e ".[viz,implicit]"
```

This installs:

- **Core**: `numpy`, `scipy`, `pyyaml`, `matplotlib`, `h5py`, `tqdm`
- **viz**: `pyvista` (3D visualisation — used by scripts with `-save` for headless image output)
- **implicit**: `scikit-image` (marching-cubes meshing for double-torus and Banchoff-Chmutov surfaces)

Important

IPOPT is not available on Pelle. The `cyipopt` package requires a system-level IPOPT installation which is not present on UPPMAX. Phase 2 refinement on the cluster uses `-method slsqp` (default) or `-method trust-constr`. The full IPOPT workflow (including exact-Hessian mode) is available on your local machine once the results are transferred.

4.6 Verify the installation

```
python -c "import numpy, scipy, h5py, yaml, pyvista; print('OK')"
```

If this prints `OK` without errors, the environment is ready.

4.7 Configure the submission scripts

The file `cluster/pelle_config.sh` is the single place where user-specific paths are set. Open it with your favourite editor:

```
nano ~/projects/surface-partition/cluster/pelle_config.sh
```

Find and update these two lines, replacing `teban66` with your own UPPMAX username:

```
REPO_DIR="/home/<your_username>/projects/surface-partition"
VENV_DIR="/home/<your_username>/venvs/surface-partition"
```

Leave all other lines unchanged — they refer to the shared project allocation and storage paths that are identical for every collaborator.

Tip

After editing `pelle_config.sh`, prevent accidental commits of your personal paths by running once:

```
cd ~/projects/surface-partition
git update-index --assume-unchanged cluster/pelle_config.sh
```

Git will then ignore local changes to that file when staging commits.

4.8 Create the SLURM log directory

The submission scripts write job output here. Create it once:

```
mkdir -p ~/projects/surface-partition/slurm_logs
```

5 Running Phase 1: Relaxation

Phase 1 performs Gamma-convergence relaxation (Projected Gradient Descent) on the surface mesh. It produces an HDF5 solution file used as input for Phase 2.

5.1 How the submission script works

`cluster/submit_relaxation.sh` generates a SLURM job script on the fly from the parameters in `cluster/pelle_config.sh` and the experiment YAML you provide, then submits it with `sbatch`. The running job:

1. Loads the Python module and activates the virtual environment.
2. Changes to `PROJECT_BASE` so results land in the shared directory.
3. Runs `scripts/find_surface_partition.py` with your config.

Results are written to `/proj/snic2020-15-36/private/LINKED_LST_MANIFOLD/results/`.

5.2 Basic usage

From inside the repository directory:

```
cd ~/projects/surface-partition
bash cluster/submit_relaxation.sh --config parameters/torus_10part.yaml
```

5.3 Available options

Option	Default	Description
`-config <yaml>`	(required)	Experiment YAML file
`-time <HH:MM:SS>`	`12:00:00`	Wall-clock time limit
`-cpus <N>`	`4`	Number of CPU cores
`-mem <XG>`	`16G`	Memory per job
`-resume-from <h5>`	—	Warm-start from a prior solution
`-solution-dir <dir>`	—	Override output directory
`-dry-run`	—	Print SLURM script without submitting

5.4 Example commands

Torus, 10 partitions (default config):

```
bash cluster/submit_relaxation.sh --config parameters/torus_10part.yaml
```

Ellipsoid, 6 partitions with longer time limit:

```
bash cluster/submit_relaxation.sh \
    --config parameters/ellipsoid_6part.yaml \
    --time 24:00:00 --cpus 8
```

Double torus, 10 partitions (implicit surface):

```
bash cluster/submit_relaxation.sh \
    --config parameters/double_torus_10part.yaml
```

Warm-start from a prior solution at a higher refinement level:

```
bash cluster/submit_relaxation.sh \
    --config parameters/torus_10part.yaml \
```

```
    --resume-from /proj/snic2020-15-36/private/LINKED_LST_MANIFOLD/results/ \
        run_20260510_120000_.../solution/surface_....h5
```

Preview the SLURM script without submitting:

```
bash cluster/submit_relaxation.sh \
    --config parameters/torus_10part.yaml --dry-run
```

5.5 Monitoring your job

```
# List all your running and queued jobs
squeue -u <your_username>

# Watch the queue (refreshes every 5 seconds)
watch -n 5 squeue -u <your_username>

# Cancel a job
scancel <job_id>

# Show detailed accounting for a finished job
sacct -j <job_id> --format=JobID,Elapsed,CPUTime,MaxRSS,State

# UPPMAX convenience tool for detailed job info
jobinfo <job_id>
```

5.6 Reading the job output

SLURM captures `stdout` and `stderr` in the log directory:

```
ls ~/projects/surface-partition/slurm_logs/
# Files are named: <job_name>_<job_id>.out  and  <job_name>_<job_id>.err

# Follow the live output of a running job
tail -f ~/projects/surface-partition/slurm_logs/<job_name>_<job_id>.out
```

6 Running Phase 2: Perimeter Refinement

Phase 2 takes the HDF5 solution from Phase 1 and minimises the total boundary perimeter subject to equal-area constraints. It requires a completed Phase 1 run.

6.1 Locating the Phase 1 solution

After Phase 1 finishes, find the solution file:

```
ls /proj/snic2020-15-36/private/LINKED_LST_MANIFOLD/results/
# Each run is in a directory named:
# run_<timestamp>_surf<surface>_npart<N>_..._lam<lambda>_seed<seed>/

ls /proj/snic2020-15-36/private/LINKED_LST_MANIFOLD/results/ \
   run_<your_run>/solution/
# The HDF5 file is: surface_part<N>_surf<surface>_..._<timestamp>.h5
```

6.2 Basic usage

```
bash cluster/submit_refinement.sh \
    --solution /proj/snic2020-15-36/private/LINKED_LST_MANIFOLD/results/ \
```

```
        run_<your_run>/solution/surface_....h5 \
    --config parameters/torus_10part.yaml
```

6.3 Available options

Option	Default	Description
`-solution <h5>`	(required)	Phase 1 solution or checkpoint
`-config <yaml>`	(required)	Experiment YAML
`-time <HH:MM:SS>`	`24:00:00`	Wall-clock time limit
`-cpus <N>`	`4`	CPU cores
`-mem <XG>`	`16G`	Memory per job
`-method <name>`	`slsqp`	Solver: `slsqp` or `trust-constr`
`-max-iterations <N>`	from YAML	Migration loop iterations
`-max-opt-iter <N>`	from YAML	Solver iterations per loop step
`-tolerance <val>`	from YAML	Solver convergence tolerance
`-boundary-tol <val>`	from YAML	Type 1 migration threshold
`-lbfgs-memory <N>`	from YAML	L-BFGS history length
`-save-iterations`	—	Save every intermediate checkpoint
`-best-iterate`	—	Keep best-perimeter iterate
`-allow-partial`	—	Continue past constraint violations
`-dry-run`	—	Print script without submitting

6.4 Example commands

Standard run using config defaults (SLSQP):

```
bash cluster/submit_refinement.sh \
    --solution /proj/.../results/run_.../solution/surface_....h5 \
    --config parameters/torus_10part.yaml
```

Trust-region constrained solver, 20 iterations:

```
bash cluster/submit_refinement.sh \
    --solution /proj/.../results/run_.../solution/surface_....h5 \
    --config parameters/torus_10part.yaml \
    --method trust-constr --max-iterations 20
```

Resume from a checkpoint (auto-detected by the pipeline):

```
bash cluster/submit_refinement.sh \
    --solution /proj/.../results/run_.../refinement/slsqp_btol0.001/ \
        iteration_003_20260510_140000.h5 \
    --config parameters/torus_10part.yaml
```

Note

Phase 2 refinement campaigns are stored under `results/run.../refinement/` inside a subdirectory named by solver and tolerances, for example `slsqp_btol0.001/` or `trust-constr_btol0.001_lbfgs30/`. A `refinement.yaml` config snapshot and `refinement.log` are written inside each campaign directory.

7 Parameter Sweeps

The sweep tool submits a grid (or paired) array of Phase 1 jobs in one command, collects results into a shared index, and produces analysis plots.

7.1 Sweep specification files

Sweep specs live under `sweep/parameters/`. Each YAML file defines the surface, partitions, and the parameters to vary. Examples:

- `sweep/parameters/sweep_torus_lambda.yaml` — grid sweep over λ values and random seeds for the torus.
- `sweep/parameters/sweep_double_torus_lambda.yaml` — grouped grid sweep over λ and mesh resolution.

7.2 Submitting a sweep

```
# Preview: see how many jobs will be submitted (no jobs submitted)
bash cluster/submit_sweep.sh \
    --sweep sweep/parameters/sweep_torus_lambda.yaml --dry-run

# Submit all combinations as separate SLURM jobs
bash cluster/submit_sweep.sh \
    --sweep sweep/parameters/sweep_torus_lambda.yaml

# Submit with a 24-hour time limit and auto-collect when done
bash cluster/submit_sweep.sh \
    --sweep sweep/parameters/sweep_torus_lambda.yaml \
    --time 24:00:00 --auto-collect
```

When `-auto-collect` is given, a final SLURM job is submitted with a dependency on all sweep jobs. It runs automatically after the last run finishes, scans the results, and writes `experiment_index.yaml` plus summary CSV files.

7.3 Collecting results manually

If you did not use `-auto-collect`, run the collector once all jobs have finished:

```
# Recommended: index + matplotlib analysis plots (no PyVista screenshots)
python sweep/parameter_sweep.py \
    --sweep sweep/parameters/sweep_torus_lambda.yaml \
    --mode collect --collect-skip-screenshots \
    --output-dir /proj/snic2020-15-36/private/LINKED_LST_MANIFOLD/results

# Index and CSV only (fastest):
python sweep/parameter_sweep.py \
    --sweep sweep/parameters/sweep_torus_lambda.yaml \
    --mode collect --collect-metadata-only \
    --output-dir /proj/snic2020-15-36/private/LINKED_LST_MANIFOLD/results
```

Note

The collector must be run from inside the repository:

```
cd ~/projects/surface-partition
source ~/venvs/surface-partition/bin/activate
module load Python/3.11.5-GCCcore-13.3.0
```

You can run the collector interactively on a login node for short sweeps; for sweeps with many runs or large meshes, submit it as a job instead.

8 Output Directory Structure

8.1 Home directory (SLURM logs)

```
~/projects/surface-partition/
+-- slurm_logs/
    |-- relax_torus_npart10_..._<job_id>.out
    |-- relax_torus_npart10_..._<job_id>.err
    +-- ...
```

These files contain the console output of each SLURM job. The `.out` file mirrors the Python logger output; `.err` captures unexpected errors.

8.2 Shared project directory (run results)

All computation results go here:

```
/proj/snic2020-15-36/private/LINKED_LST_MANIFOLD/results/
```

8.3 Per-run directory layout

Each Phase 1 run produces a self-contained directory:

```
results/
+-- run_<timestamp>_surf<surface>_npart<N>_..._lam<lambda>_seed<seed>/
    |-- experiment.yaml          # verbatim copy of the input config
    |-- solution/
    |   |-- surface_part<N>_..._<timestamp>.h5   # Phase 1 output
    |   +-- metadata.yaml
    |-- traces/                  # PGD internal data per refinement level
    |-- refinement/
    |   |-- slsqp_btol0.001/     # one subdirectory per Phase 2 campaign
    |   |   |-- iteration_001_<timestamp>.h5
    |   |   |-- refinement.yaml  # config snapshot for reproducibility
    |   |   +-- refinement.log
    |   +-- trust-constr_btol0.001_lbfgs30/
    |       +-- ...
    |-- analysis/                # energy and constraint plots
    +-- logs/
        +-- relaxation.log
```

8.4 Experiment directory layout (parameter sweeps)

Sweep runs are grouped by surface and partition count:

```
results/
+-- torus_npart10/
    |-- experiment_index.yaml              # auto-maintained index of all runs
    |-- run_<timestamp>_..._lam0.5_seed42/
    |   +-- ...                            # same per-run layout as above
    |-- run_<timestamp>_..._lam1.0_seed42/
    |   +-- ...
    |-- sweeps/                            # sweep provenance records
```

```
    |   |-- <timestamp>_<sweep_name>.yaml
    |   +-- <timestamp>_<sweep_name>_summary.csv
    +-- analysis/                          # experiment-wide comparison plots
        |-- heatmap_perimeter.png
        |-- line_perimeter.png
        +-- convergence_overlay.png
```

The `experiment_index.yaml` file is the central record. It lists every run with its parameters, status, and key metrics (perimeter, final energy, mesh size, convergence flag). Perimeter is the primary comparison metric because it is resolution-independent.

9 Retrieving and Analysing Results

9.1 Transferring files to your local machine

Use `rsync` to copy results efficiently (only transfers changed files):

```
# Copy a single run directory
rsync -avz --progress \
    <your_username>@pelle.uppmax.uu.se:/proj/snic2020-15-36/private/ \
    LINKED_LST_MANIFOLD/results/run_<your_run>/ \
    ~/local-results/run_<your_run>/

# Copy an entire experiment directory (e.g. torus_npart10)
rsync -avz --progress \
    <your_username>@pelle.uppmax.uu.se:/proj/snic2020-15-36/private/ \
    LINKED_LST_MANIFOLD/results/torus_npart10/ \
    ~/local-results/torus_npart10/

# Transfer only solution HDF5 files (skip large trace data)
rsync -avz --progress --include="*.h5" --include="*/" \
    --exclude="traces/**" \
    <your_username>@pelle.uppmax.uu.se:/proj/snic2020-15-36/private/ \
    LINKED_LST_MANIFOLD/results/run_<your_run>/ \
    ~/local-results/run_<your_run>/
```

Note

`rsync` preserves directory structure and skips files that have not changed since the last transfer. Run it again at any time to pull updates without re-downloading everything.

9.2 Local analysis (on your laptop)

Once the files are on your local machine, activate your local environment and run the analysis scripts from the repository root:

```
# Activate pyenv environment (see .python-version)
pyenv activate surface-partition

# Analyse a single run
python scripts/optimization_analyzer.py \
    --results-dir ~/local-results/run_<your_run>/

# Sweep-wide analysis (reads experiment_index.yaml)
python sweep/sweep_analyzer.py \
    --experiment-dir ~/local-results/torus_npart10/
```

```
# Timing analysis (only for runs with --profile flag)
python sweep/timing_analyzer.py \
    --experiment-dir ~/local-results/torus_npart10/
```

9.3 Visualising partitions (requires PyVista)

Visualisation scripts require a local PyVista install (`pip install -e ".[viz]"`). On the cluster they can only produce saved images (no interactive window).

```
# Fast production viewer (recommended for fine meshes)
python scripts/visualize_partition_fast.py \
    --solution ~/local-results/run_.../solution/surface_....h5

# Visualise a Phase 2 refined checkpoint
python scripts/visualize_partition_fast.py \
    --solution ~/local-results/run_.../refinement/slsqp_btol0.001/ \
        iteration_005_<timestamp>.h5 \
    --show-steiner

# Save an image instead of opening an interactive window
python scripts/visualize_partition_fast.py \
    --solution ... --save partition.png
```

10 Useful SLURM Commands

Command	What it does
`squeue -u <username>`	List your running and pending jobs
`squeue -u <username> -t R`	List only running jobs
`scancel <job_id>`	Cancel a job
`scancel -u <username>`	Cancel *all* your jobs
`sacct -j <job_id> -format=Elapsed,MaxRSS,State`	Job runtime, memory, and exit state
`jobinfo <job_id>`	UPPMAX-specific detailed job info
`projinfo`	Show your allocation usage
`uquota`	Show your home directory disk usage

11 Troubleshooting

"Virtual environment not found" at job startup. The job cannot find `VENV_DIR`. Check that you:

- Updated `VENV_DIR` in `cluster/pelle_config.sh` to your own username (Section 4.7).
- Ran the setup steps in Section 4 on Pelle (not just locally).
- Did not delete or move the venv directory.

"ModuleNotFoundError: No module named ..." in the job log. The Python module was loaded but the package is not in the venv. Re-run the install step:

```
module load Python/3.11.5-GCCcore-13.3.0
source ~/venvs/surface-partition/bin/activate
cd ~/projects/surface-partition
pip install -e ".[viz,implicit]"
```

Job fails immediately with exit code 1. Check the `.err` log file in `slurm_logs/`. Common causes:

- Config YAML file path not found — use an absolute path or ensure the path is correct relative to `PROJECT_BASE`.
- HDF5 solution file not found — check the `-solution` path.
- Permission denied on `/proj/snic2020-15-36/` — confirm with the PI that you have been added to the storage project.

"Permission denied" writing to the results directory. You need to be a member of the storage allocation `snic2020-15-36`. Ask the PI to add you via SUPR.

The job is queued for a long time. Check allocation usage with `projinfo`. If the core-hour balance is exhausted the queue priority drops. Reduce `-cpus` and `-time` to the minimum needed for your run.

I cannot find my results after the job finishes. Results are written to `PROJECT_BASE/results/`, which is `/proj/snic2020-15-36/private/LINKED_LST_MANIFOLD/results/` — not to your home directory. Use `ls` there and look for a `run_<timestamp>_...` directory with a recent timestamp.

A pelle_config.sh Quick Reference

The table below lists every variable in `cluster/pelle_config.sh` and whether collaborators need to edit it.

Variable	Edit?	Description
`PROJECT_ID`	No	SLURM allocation ID
`PROJECT_BASE`	No	Shared data root on `/proj/`
`RESULTS_BASE`	No	Derived from `PROJECT_BASE`
`REPO_DIR`	**Yes**	Absolute path to your clone of the repo
`PYTHON_MODULE`	No*	Module string for `module load`
`VENV_DIR`	**Yes**	Absolute path to your Python venv
`DEFAULT_TIME`	No	Default SLURM time limit
`DEFAULT_CPUS`	No	Default CPU count
`DEFAULT_MEM`	No	Default memory per job

*Only edit `PYTHON_MODULE` if the module name changes on UPPMAX, e.g. after a system upgrade.

B Available Experiment Configurations

The `parameters/` directory ships with four ready-to-use experiment configs:

Config file	Surface	Partitions
`torus_10part.yaml`	Torus ($R = 1$, $r = 0.6$)	10
`ellipsoid_6part.yaml`	Ellipsoid	6
`double_torus_10part.yaml`	Double torus (implicit)	10
`banchoff_chmutov_12part.yaml`	Banchoff-Chmutov order 4 (implicit)	12

The double-torus and Banchoff-Chmutov configs use implicit surface meshing (marching cubes) and require the `.[implicit]` dependency group, which is included in the cluster installation above.